

PROJETO E ANÁLISE DE ALGORITMOS – 01A – 2022.2

[Página inicial](#)

[Meus cursos](#)

[PROJETO E ANÁLISE DE ALGORITMOS – 01A – 2022.2](#)

[Tópico 9](#)

[Notas de aula: busca em largura \(BFS\)](#)

Notas de aula: busca em largura (BFS)

Grafos são estruturas mais complexas que vetores, listas e árvores. Precisamos de métodos para explorar os vértices e arestas. Veremos busca em largura e em profundidade, suas propriedades e algumas aplicações.

r	s	t	u
oo ----	0	oo ----	oo
		/	/
		/	/
		/	/
		/	/
		/	/
oo	oo ----	oo ----	oo
v	w	x	y

Q | s |

0

r	(s)	t	u
1 xxxxx	0	oo ----	oo
	x	/	/
	x	/	/
	x	/	/
	x	/	/
	x	/	/
oo	1 ----	oo ----	oo
v	w	x	y

Q | w | r |

1 1

r	(s)	t	u
1 xxxxx	0	2 ----	oo
	x	x	/
	x	x	/
	x	x	/
	x	x	/
	x	x	/
oo	1 xxxxx	2 ----	oo
v	(w)	x	y

Q | r | t | x |

1 2 2

(r)	(s)	t	u
1 xxxxx	0	2 ----	oo
x	x	x	/
x	x	x	/
x	x	x	/
x	x	x	/
x	x	x	/
2	1 xxxxx	2 ----	oo
v	(w)	x	y

Q | t | x | v |

2 2 2

(r)	(s)	(t)	u
1 xxxxx	0	2 xxxxx	3
x	x	x	/
x	x	x	/
x	x	x	/
x	x	x	/
x	x	x	/
2	1 xxxxx	2 ----	oo
v	(w)	x	y

Q | x | v | u |

2 2 3

(r)	(s)	(t)	u
1 xxxxx	0	2 xxxxx	3
x	x	x	/
x	x	x	/
x	x	x	/
x	x	x	/
x	x	x	/
2	1 xxxxx	2 xxxxx	3
v	(w)	(x)	y

Q | v | u | y |

2 3 3

(r)	(s)	(t)	(u)
1 xxxxx	0	2 xxxxx	3
x	x	x	/
x	x	x	/
x	x	x	/
x	x	x	/
x	x	x	/
2	1 xxxxx	2 xxxxx	3
(v)	(w)	(x)	(y)

Q VAZIA

```
BFS(G,s):
  // INICIALIZAÇÃO
  1 para cada vértice u de G (excluindo o s)
  2   cor[u] <- branco; d[u] <- oo; pi[u] <- NULO
  3 cor[s] <- cinza; d[s] <- 0; pi[s] <- NULO

  4 declare uma fila vazia Q
  5 insira s na fila Q
  6 enquanto Q não é vazia
  7   remova o próximo elem. da fila Q e o armazene em u
  8   para cada vértice v em Adj[u]
  9     se cor[v] = branco
10       cor[v] <- cinza; d[v] <- d[u] + 1; pi[v] <- u
11       insira v na fila Q
12   cor[u] <- preto
```

Complexidade da busca em largura (grafo em lista de adjacência):

- Inicialização: $O(V)$.
- Declaração de fila vazia: $O(1)$.
- Inserção ou remoção de um elemento na fila: $O(1)$.
- Cada vértice passa pela fila Q no máximo uma vez, pois só é inserido se for branco, e fica cinza quando é inserido.
- Quando um vértice sai da lista, sua lista de adjacência é percorrida. Portanto, no pior caso cada aresta é visitada uma vez.
- Conclusão: complexidade da busca em largura vale $O(V + E)$.

Esta foi uma análise agregada: consideramos o total de execuções das linhas 8-11, e não o pior caso de cada linha (que vale $O(V)$). Portanto, em uma análise grosseira teríamos concluído que o algoritmo é $O(V^2)$. Isto está correto, mas a complexidade $O(V + E)$ é mais informativa.

BFS(G,s):	
// INICIALIZAÇÃO	
para cada vértice u de G (excluindo o s)	$O(V)$
cor[u] <- branco; d[u] <- oo; pi[u] <- NULO	$O(V)$
cor[s] <- cinza; d[s] <- 0; pi[s] <- NULO	$O(1)$
declare uma fila vazia Q	$O(1)$
insira s na fila Q	$O(1)$
enquanto Q não é vazia	$O(V)$
remova o próximo elem. da fila Q e o armazene em u	$O(V)$
para cada vértice v em Adj[u]	$O(V+E)$
se cor[v] = branco	$O(E)$
cor[v] <- cinza; d[v] <- d[u] + 1; pi[v] <- u	$O(E)$
insira v na fila Q	$O(E)$
cor[u] <- preto	$O(V)$

Qual a complexidade da busca em largura se o grafo está representado por matriz de adjacência? Para percorrer os adjacentes de v nas linhas 8-11 gastamos $O(V)$ para cada vértice v , tornando assim o algoritmo $O(V^2 + E) = O(V^2)$.

BFS(G,s):	
// INICIALIZAÇÃO	
para cada vértice u de G (excluindo o s)	$O(V)$
cor[u] <- branco; d[u] <- oo; pi[u] <- NULO	$O(V)$
cor[s] <- cinza; d[s] <- 0; pi[s] <- NULO	$O(1)$
declare uma fila vazia Q	$O(1)$
insira s na fila Q	$O(1)$
enquanto Q não é vazia	$O(V)$
remova o próximo elem. da fila Q e o armazene em u	$O(V)$
para cada vértice v	$O(V^2)$
se M[u,v]	$O(V^2)$
se cor[v] = branco	$O(E)$
cor[v] <- cinza; d[v] <- d[u] + 1; pi[v] <- u	$O(E)$
insira v na fila Q	$O(E)$
cor[u] <- preto	$O(V)$

Comparação das complexidades:

Grafo denso: $E = O(V^2)$. Neste caso, $O(V + E) = O(V + V^2) = O(V^2)$. Ou seja, as duas representações têm a mesma complexidade.

Grafo esparso: $E = O(V)$. Neste caso, $O(V + E) = O(V + V) = O(V)$. Ou seja, é melhor representar o grafo com lista de adjacência.

Propriedades

Seja $dist(v)$ o menor número de arestas de s até v . Note que isso é uma característica do grafo, e não depende do algoritmo.

“**Lema:** Para toda aresta $\{u, v\}$, temos que $|dist(v) - dist(u)| \leq 1$.”

Prova: Como o caminho mais curto de s até u , juntamente com a aresta $\{u, v\}$, forma um caminho de s até v , não precisamos gastar mais de $dist(u) + 1$ arestas para ir de s até v . O mesmo vale para o caminho de s até u passando por v .

“**Lema:** Durante toda execução temos que $d[v] \geq dist(v)$ para todo vértice v .”

Prova: Por indução no número de inserções de vértices em Q .

Caso base: após a inicialização, temos $d[s] = dist(s) = 0$ e $d[v] = \infty$ p/ todo vértice v diferente de s .

Hipótese de indução: em uma dada iteração, $d[v] \geq dist(v)$ p/ todo vértice.

Identificamos o próximo vértice v a ser inserido em Q ao explorar os adjacentes a u . Por H.I., $d[u] \geq dist(u)$. Então,

$$d[v] = d[u] + 1 \geq dist(u) + 1 \geq dist(v).$$

“**Lema:** Durante toda execução os vértices na fila Q tem no máximo dois valores consecutivos para d , e estão dispostos em ordem crescente.”

Prova: Por indução no número de inserções de vértices em Q .

Caso base: inicialmente a fila tem apenas o elemento s , com $d[s] = 0$.

Hipótese de indução: em um dado instante temos apenas o valor k , ou temos também o valor $k + 1$ ocorrendo depois dos elementos com valor k .

Quando o próximo vértice v sai do início de Q (com $d[v] = k$), se algum vértice u não explorado e adjacente a v for encontrado, ele entrará no final de Q com $d[u] = k + 1$.

“**Teorema:** Ao final do algoritmo temos $d[v] = dist(v)$ para todo vértice v .”

Prova: Vamos provar por indução em $dist(v)$.

Caso base: s é o único vértice v com $dist(v) = d[v] = 0$.

Hipótese de indução: suponha que $d[u] = dist(u)$ p/ todo u com $dist(u) < k$.

Queremos provar que $d[v] = k$ para todo vértice v com $dist(v) = k$.

Seja v um vértice qualquer com $dist(v) = k$. Seja u o vértice que antecede v em um caminho mínimo qualquer de s até v . Então $dist(u) = k - 1$, e por hipótese, $d[u] = k - 1$.

Considere o instante que u saiu de Q .

Caso I: v é branco. Então u provocará a entrada de v em Q . Logo, $d[v] = d[u] + 1 = k$.

Caso II: v é cinza. Então v está em Q , logo tem $d[v] = k - 1$ ou $d[v] = k$. Como $d[v] \geq dist(v) = k$, temos que $d[v] = k$.

Caso III: v é preto. Então já passou pela fila, e portanto tem $d[v] \leq d[u] = k - 1$ (contradição, pois $d[v] \geq dist(v) = k$).

Árvore de caminhos mais curtos (menor número de arestas)

Os predecessores, armazenados em π , definem uma árvore com raiz no vértice inicial s , que chamaremos de Árvore de Busca em Largura (ABL). Uma árvore estabelece um único caminho simples de qualquer nó até a raiz.

“**Lema:** As arestas de uma ABL fornecem os caminhos mínimos (menor número de arestas) de qualquer vértice até o vértice inicial s .”

Prova: Basta mostrar que, para qualquer vértice u , o número de arestas na ABL de u até s é igual a $d[u] = dist(u)$. Prova por indução em $dist()$.

Caso base: o vértice inicial s está a 0 aresta dele próprio, e possui $d[s] = 0$.

Assuma que o lema vale para todo vértice v com $dist(v) < k$.

Seja u um vértice qualquer com $dist(u) = k$, e v o predecessor de u na ABL. Então, $d[v] = k - 1 = dist(v)$, e podemos aplicar a hipótese de indução em v . Ou seja, temos $k - 1$ aresta de v até s na ABL. Com a definição de v como precessor u , temos um caminho com k arestas de u até s .

Exercícios

1. Mostre que uma variável booleana seria suficiente para armazenar a cor de um vértice na BFS (ou seja, basta usar 2 cores).
- Solução: note que todos os testes verificam se o vértice é branco. Portanto, atribuir cor preta no final não tem efeito no algoritmo, pois neste momento o vértice já tem cor cinza.
1. Forneça duas listas de adjacências para um mesmo grafo que produzam duas Florestas de BP distintas, mesmo quando a BFS é iniciada em um mesmo vértice.

Solução: $V = \{a, b, c, d\}$, $E = \{(a, b), (a, c), (b, d), (c, d)\}$,
 $Adj1[a] \rightarrow b \rightarrow c$, $Adj1[b] \rightarrow a \rightarrow d$, $Adj1[c] \rightarrow a \rightarrow d$, $Adj1[d] \rightarrow b \rightarrow c$
Única diferença em $Adj2$: $Adj2[a] \rightarrow c \rightarrow b$.
Com $Adj1$ a Floresta de BP terá a aresta (b, d) , e com $Adj2$ terá a aresta (c, d) .

1. Forneça um grafo G e uma subárvore T de G de modo que T seja uma árvore de caminhos mínimos para um dado vértice u , mas T não possa ser obtida através de uma BFS iniciando em u .

Solução: $V = \{a, b, c, d, e\}$, $E = \{(a, b), (a, c), (b, d), (c, d), (c, e), (b, e)\}$, iniciando em a .
 $T = \{(a, b), (a, c), (c, e), (b, d)\}$.
Caso I: Se b é colocado na fila antes de c , então 'd' e 'e' seriam descendentes de b .
Caso II: Se c é colocado na fila antes de b , então 'd' e 'e' seriam descendentes de c .

1. Os valores $d[]$ dependem da ordem dos vértices na lista de adjacências? Justifique.

Solução: Sim. $dist()$ é uma característica do grafo (não depende da representação). Além disso, durante a demonstração de que $d[u] = dist(u)$, para todo vértice u , não usamos a ordem dos vértices na lista de adjacências.

Última atualização: quinta, 3 dez 2020, 10:57

[◀ Notas de aula: noções de teoria dos grafos](#)

Seguir para...

[Notas de aula: caminhos mais curtos com mesma origem ▶](#)